

EMERGENT PHASE 2 MVP - PRODUCTION-READY PROMPT

PROJECT CONTEXT

Platform: React Native + Expo (Pure JavaScript)

Phase 1 Status: Complete and working - 12 screens, navigation, Redux, mock data

Current Structure: Added /services folder with chatService.js for API calls

Phase 2 Focus: 7 critical MVP features for production readiness

PHASE 1 MISSING ANALYSIS

Based on Phase 1 specification vs current implementation, the following files are **missing or need verification**:

I need a ui change, there were originally 2 tabs from phase 1.

1) Chat

2) dashboard

now we need a third tab called as Issues, on clicking it, the user must be taken to total issues page. And, inside the dashboard page, take out the total issues page. And the remaining Not fixed, Fixed and Complaints boxes leading to the respective screens must exist with all functionalities.

Services Layer (ADDED - You confirmed this)

- ✓ /src/services/chatService.js - You added this manually

Additional Services Needed (From Phase 1 spec)

The Phase 1 spec mentioned mocks/apiService.js but you've created a proper services architecture. Complete the services layer:

- /src/services/apiService.js - Base API wrapper with axios configuration
- /src/services/issueService.js - Issue CRUD operations
- /src/services/complaintService.js - Complaint operations
- /src/services/dashboardService.js - Dashboard data fetching
- /src/services/authService.js - Authentication calls

Missing Utils (Critical for Phase 2)

Missing Utils (Critical for Phase 2)

Missing Utils (Critical for Phase 2)

Missing Utils (Critical for Phase 2)

- `/src/utils/permissions.js` - Camera/location permissions (CRITICAL for Phase 2)
 - `/src/utils/storage.js` - AsyncStorage wrappers
 - `/src/utils/overdue.js` - Calculate overdue days
-

PHASE 2 MVP FEATURES SPECIFICATION

FEATURE 3: OFFLINE MODE & SYNC

What It Is: App detects network status and displays offline indicator at top. No action queuing - just visual feedback.

Implementation Requirements:

- Network Status Detection using `@react-native-community/netinfo`
- Visual indicator: Message at top showing offline/online status
- When offline: Red banner at top with "No internet connection" message
- When online: Banner disappears (or brief green confirmation)

New Files to Create:

text

`/src/utils/network.js` - Network monitoring utilities

`/src/components/common/OfflineBanner.js` - Top banner component

`/src/hooks/useNetworkStatus.js` - Network status hook

Redux Enhancements:

`offlineSlice.js` (exists in Phase 1, now fully implement):

- State: `{isOnline: boolean, lastChecked: timestamp}`
- Actions: `setOnlineStatus(isOnline)`
- No Thunks needed - just status tracking

Offline Banner Behavior:

- Appears on ALL screens when offline
- Positioned at absolute top (above navigation)
- Red background: #ef4444
- White text with WiFi-off icon: "⚠ No internet connection"
- Auto-shows when connection lost
- Auto-hides when connection restored
- Optional: Brief green banner "✓ Back online" (2 second auto-dismiss)

Implementation in `useNetworkStatus.js`:

javascript

```
// Subscribe to network state changes  
// Dispatch setOnlineStatus(isOnline) to Redux  
// Banner component reads from Redux and conditionally renders
```

Implementation in `OfflineBanner.js`:

javascript

```
// Read isOnline from Redux  
// If offline: Show red banner at top  
// If just came online: Show green banner for 2 seconds, then hide  
// Use absolute positioning: top: 0, zIndex: 9999
```

Behavior:

- When offline: All API calls fail gracefully with error messages
- No queuing: User sees real-time status, must retry actions manually when back online
- Data display: Show cached data with timestamp "Last updated X minutes ago"

Testing:

Test Scenario 1: Go Offline

- User browsing Issue List
- Turn on airplane mode
- Red banner appears: "⚠ No internet connection"

- User can still view cached data
 - Any API action (refresh, create) shows error: "No internet connection"
 - Test Scenario 2: Come Back Online
 - User offline, red banner showing
 - Turn off airplane mode
 - Red banner disappears
 - Optional: Green "✓ Back online" shows for 2 seconds
 - User can now perform actions normally
-

FEATURE 6: PULL-TO-REFRESH

What It Is: User pulls down from top of list to refresh data. Standard mobile UX pattern.

Screens That Need It:

- DashboardScreen
- IssueListScreen
- NotFixedIssuesListScreen
- FixedIssuesListScreen
- ComplaintsListScreen

Implementation:

Use React Native's `RefreshControl` component wrapped around `ScrollView/FlatList`.

Props:

- `refreshing`: boolean state
- `onRefresh`: function to fetch fresh data
- `colors`: [`#2563eb`] (theme primary color)
- `tintColor`: `#2563eb` (iOS)

Behavior:

1. User at top of list
2. Swipes down gesture
3. Spinner appears following finger
4. User releases
5. Spinner stays at top, data fetches
6. Redux thunk dispatches (e.g., `fetchIssues()`)
7. Data updates in state
8. Spinner disappears

9. List shows fresh data

Offline Handling:

If offline when pull-to-refresh triggered:

- Show spinner
- Toast: "Can't refresh while offline"
- No action taken

Cooldown Logic:

Prevent spam refreshing:

- If last refresh < 5 seconds ago
- Ignore gesture or show toast: "Just refreshed. Wait a moment."

Files to Modify:

- All list screens - Add `RefreshControl` to `ScrollView/FlatList`
- Each screen's refresh handler dispatches appropriate Redux thunk
- Add `refreshing` local state to each screen

Testing:

Test Scenario 1: Successful Refresh

- User on Issue List
- Pull down from top
- Spinner appears
- Release
- Spinner spins for 1-2 seconds
- Fresh data loads
- Spinner disappears
- List updates

Test Scenario 2: Refresh While Offline

- User on Dashboard
- Turn on airplane mode (offline banner shows)
- Pull to refresh
- No spinner appears
- Toast: "Can't refresh while offline"

Test Scenario 3: Refresh Cooldown

- User pulls to refresh
- Refresh completes
- Immediately pull again (within 5 seconds)

- **Toast: "Just refreshed. Wait a moment."**
- **No refresh triggered**

FEATURE 5: ADVANCED FILTERS & SEARCH

What It Is: Enhanced search with debouncing and comprehensive filter modal for issues/complaints.

Search Enhancements:

- Debounced search (500ms delay after typing stops)
- Multi-field search: Issue ID, title, description, site name, solver name
- Search result highlighting (matching text in yellow)
- Search history (last 5 searches stored)
- Clear button (X) when text entered

New Files:

- `/src/components/modals/FilterModal.js` - Full-screen filter modal
- `/src/components/common/FilterChip.js` - Active filter chip display
- `/src/components/common/SearchBar.js` - Enhanced search component
- `/src/hooks/useDebounce.js` - Debounce hook for search

Filter Modal Design:

Full-screen modal with sections:

Filter Type	Options
Status	Multi-select checkboxes: Open, Assigned, In Progress, Completed, etc.
Priority	Radio buttons: All, High, Medium, Low
Site	Multi-select dropdown (role-restricted)
Date Range	Start/End date pickers with presets (Today, This Week, This Month)
Deadline	Radio: All, Overdue only, Due today, Due this week
Solver	Dropdown (Managers/Supervisors only)

Table 1: Filter Modal Options

Filter Chips Display:

After applying filters, show chips below search bar:

- Blue rounded rectangles
 - Filter name and value
 - X icon to remove individual filter
 - "Clear All" button if multiple active
- Example: "Status: Open" [x] "Priority: High" [x] "Site: Vepery" [x]

Redux Enhancements:

issuesSlice.js additions:

- State: Add filters: {status: [], priority: null, siteIds: [], dateRange: {}, ...}
- Action: applyFilters(filters)
- Action: clearFilters()
- Selector: selectFilteredAndSearchedIssues - combines role filter + user filters + search

Filter Logic:

All filters use AND logic:

Role-based filter (Manager: all, Supervisor: sites, Solver: assigned)

→ User's custom filters (status, priority, site, date)

→ Search text filter

→ Sort (overdue first, priority, date)

→ Display

FEATURE 4: NETWORK RETRY LOGIC

What It Is: Automatically retry failed requests with exponential backoff. Handles temporary network issues gracefully.

Retry-Worthy Failures:

- Network timeout (no response after 30 seconds)

- Connection refused
- Device offline
- 5xx server errors

Don't Retry:

- 401 Unauthorized (logout user)
- 404 Not Found
- 400 Bad Request
- User cancelled

Exponential Backoff Strategy:

Attempt	Wait Time
1	Immediate
2	2 seconds
3	4 seconds
4	8 seconds

Max 4 attempts, then show error message.

Implementation Files:

text

```
/src/utils/network.js - Add retryWithBackoff(fn, options)  
function
```

```
/src/services/apiService.js - Wrap all API calls with retry  
logic
```


Retry Wrapper Function:

javascript

```
retryWithBackoff(apiCall, {  
  maxAttempts: 4,  
  baseDelay: 2000,  
  maxDelay: 16000,  
  shouldRetry: (error) => /* check if retry-worthy */  
})
```

UI Feedback:

- Toast notification: "Connection lost. Retrying... (Attempt 2 of 4)"
- Button shows spinner with text "Retrying..."
- Button disabled during retry
- Form data preserved if retry fails

Different Configs:

- GET requests: Fast retry, more attempts (max 4)
- POST requests: Slower retry, fewer attempts (max 3)
- File uploads: Much slower retry, more attempts (max 4)
- Login: Only 2 retries

After All Retries Fail:

- Show error toast: "Connection failed. Please try again."
- Return button to normal state
- Form data preserved
- User can manually retry by tapping button again
- No queuing - just clear error feedback

Testing:

Test Scenario 1: Successful Retry

- User taps "Create Issue"
- Mock API simulates failure
- Toast: "Connection lost. Retrying... (Attempt 1 of 4)"
- Wait 2 seconds

- Second attempt succeeds
- Toast: "✓ Issue created"
- UI updates

Test Scenario 2: All Retries Fail

- User taps "Mark Complete"
- Mock API fails 4 times
- Toasts show: Attempt 1, 2, 3, 4
- After 4th failure: "Connection failed. Please try again."
- Button returns to normal state
- User can tap again to retry manually

Test Scenario 3: Form Preservation

- User fills long form
- Taps submit
- All retries fail
- Form data still filled
- Error message shown
- User can review and submit again

FEATURE 7: PHOTO UPLOAD WITH CAMERA INTEGRATION

What It Is: Real camera integration for supervisors and problem solvers. Photos uploaded when creating issues or marking complete.

New Dependencies:

- expo-camera - Camera access
- expo-image-picker - Gallery access (already in Phase 1)

Camera Button Location:

In ChatbotScreen input area:

- Left side of text input
- Visible for Supervisor and Problem Solver only
- Hidden for Manager: `user.role !== 'manager'`

Photo Upload Flow:

1. User taps camera icon in chat
2. Action sheet appears: "Take Photo" | "Choose from Gallery" | "Cancel"
3. If "Take Photo" selected:
 - Check camera permission (see Feature 8)

- If granted: Open camera
 - User takes photo
 - Preview screen with "Retake" and "Use Photo" buttons
 - User taps "Use Photo"
4. If "Choose from Gallery" selected:
 - Check photo library permission
 - Open picker
 - User selects photo
 5. Photo validation:
 - Max size: 10MB (show error if larger - compression in Phase 3)
 - Allowed types: JPEG, PNG only
 6. Display preview in chat:
 - Thumbnail in chat message
 - "Uploading..." text
 - Progress bar (0-100%)
 7. Simulate upload (Phase 2 mock):
 - Progress bar fills over 2-3 seconds
 - Store photo in AsyncStorage
 - Generate mock URL
 8. Upload complete:
 - ✓ badge: "Photo Uploaded"
 - If fix photo: Trigger location capture (Feature 8)

New Files:

- `/src/components/chat/PhotoUploadButton.js` - Camera icon button
- `/src/components/modals/CameraModal.js` - Camera interface
- `/src/components/chat/PhotoPreview.js` - Photo preview in chat
- `/src/hooks/usePhotoUpload.js` - Photo upload logic hook (exists in Phase 1, enhance)
- `/src/utils/photoStorage.js` - Local photo storage utilities

Context-Aware Upload:

When navigating to chat from specific action, pass context:

- From "Upload Fix Photo" button: `{action: 'upload_fix', issueId: 123}`
- From "Raise Complaint" button: `{action: 'raise_complaint', issueId: 123}`

- From issue creation: `{action: 'create_issue', tempIssueId: 'temp_1'}`

Chat detects context and shows appropriate prompt.

Multiple Photos:

- User can upload up to 3 photos per issue
- Each shows separate progress bar
- Number indicator: "Photo 1 of 3", "Photo 2 of 3"
- All stored with same issue/complaint ID

Error Handling:

Error	Action
Upload fails	Show "Upload failed. Retry?" button, queue for offline sync
File too large	"Image too large. Select smaller image." (Phase 3: compress)
Invalid format	"Please select an image (JPEG or PNG)"
Permission denied	Show modal with "Open Settings" button

Table 3: Photo Upload Error Handling

FEATURE 8: LOCATION PERMISSIONS & GPS CAPTURE

What It Is: Capture GPS coordinates when problem solver uploads fix photo. Verifies solver was at the site. Location permission required.

When Location is Required:

MANDATORY when Problem Solver marks issue complete. Process:

1. Solver uploads fix photo (Feature 7)
2. Photo upload completes
3. App checks location permission
4. If not granted: Show LocationPermissionModal
5. If granted: Capture GPS automatically
6. Validate location against site coordinates
7. Store latitude, longitude with photo

New Files:

- `/src/components/modals/LocationPermissionModal.js` - Permission request modal

- `/src/utils/location.js` - GPS utilities (verify this exists from Phase 1)
- `/src/hooks/useLocation.js` - Location hook (verify this exists from Phase 1)

LocationPermissionModal Design:

Full-screen blocking modal:

- Large location pin icon (60px)
- Title: "📍 Location Required"
- Message: "We need your location to verify you completed this issue at the correct site. Your location is only recorded when closing issues."
- Two buttons:
 - Primary (blue): "Enable Location"
 - Secondary (gray): "Cancel"
- Cannot dismiss by tapping outside

Permission Flow:

1. User taps "Enable Location"
2. Request permission from OS (`expo-location`)
3. OS shows system dialog
4. If user allows:
 - Permission granted
 - Modal closes
 - GPS capture begins
5. If user denies:
 - Show error modal: "Location Permission Denied"
 - Message: "To mark issues complete, enable location in Settings > Apps > AI Ops > Permissions"
 - Buttons: "Open Settings" (deep links) | "Cancel"

GPS Capture:

After permission granted:

1. Use `expo-location.getCurrentPositionAsync()` with options:
 - Accuracy: High (GPS-level)
 - Timeout: 30 seconds
 - maximumAge: 0 (fresh, not cached)
2. Returns: `{latitude: 13.0827, longitude: 80.2707, accuracy: 10}`
3. Show success badge in chat: "✓ Location Captured"
4. Verify location against site coordinates

Location Verification:

Calculate distance using haversine formula:

- Distance < 500m: ✓ "Vepery Site verified"
- Distance 500m - 1km: ⚠️"Location seems far from site"
- Distance > 1km: ✖️Alert: "You don't appear to be at the site. Continue anyway?"
 - Buttons: "Yes, Proceed" | "Cancel"
 - Allow proceeding (user can explain later)

Location Utilities (utils/location.js):

- requestLocationPermission() - Request GPS permission
- getCurrentPosition(options) - Get coordinates
- calculateDistance(point1, point2) - Haversine formula
- verifySiteLocation(captured, site) - Check if within radius

Storage:

Store location with photo in issueImages:

```
{
  "id": 1,
  "issue_id": 123,
  "image_url": "...",
  "image_type": "AFTER",
  "location_lat": 13.0827,
  "location_lon": 80.2707,
  "location_accuracy": 10,
  "location_captured_at": "2026-02-16T15:30:00"
}
```

Error Handling:

Error	Action
Timeout (>30s)	"Couldn't get location. Ensure GPS enabled." + Retry button
GPS disabled	"Turn on location services in settings." + Open Settings
Low accuracy (>100m)	Warning: "Location accuracy low. Wait or proceed anyway?"
Permission denied	Show settings modal with "Open Settings" button

Table 4: Location Error Handling

FEATURE 2: Internal Navigation (IN-APP NOTIFICATIONS ONLY)

What It Is: When user taps notification banner inside the app, navigate directly to the relevant screen (Issue Detail, Complaint Detail).

Scope for MVP: In-app notifications only, NOT external/push notifications.

Implementation:

- Notification banner in ChatbotScreen (from Phase 1)
- When tapped: Parse notification data
- Navigate based on type:
 - Type "issue" → Navigate to IssueDetailScreen with issueId
 - Type "complaint" → Navigate to ComplaintDetailScreen with complaintId
- Highlight target item with yellow background
- Yellow fades to white over 2 seconds

Navigation Configuration:

React Navigation linking config:

```
{
  screens: {
    Dashboard: {
      screens: {
        IssueDetail: 'issue/:issueId',
        ComplaintDetail: 'complaint/:complaintId'
      }
    }
  }
}
```

Deep Link Handling:

1. User taps notification banner
2. Parse notification: `{type: 'issue', issueId: 123}`
3. Navigation dispatch:
`navigation.navigate('Dashboard', {
 screen: 'IssueDetail',
 params: { issueId: 123, highlighted: true }
})`
4. IssueDetailScreen receives `highlighted: true`
5. Apply highlight effect for 2 seconds
6. Remove highlight

Highlight Implementation:

In IssueCard or detail screen:

- Check if `params.highlighted === true`
- Apply background color: `#fef3c7` (yellow-100)
- Use `Animated` API to fade to white
- Duration: 2000ms
- After fade: Remove highlight state

Notification Banner Enhancement:

- Banner shows unread count: "🔔 3 New Notifications"
- Tappable entire banner
- `onPress`: Navigate to most recent notification's target
- Dismiss button (X): Clears banner, marks as read

Files to Modify:

- `/src/components/chat/NotificationBanner.js` - Add `onPress` navigation
- `/src/navigation/MainNavigator.js` - Add linking configuration
- `/src/screens/IssueDetailScreen.js` - Add highlight effect
- `/src/screens/ComplaintDetailScreen.js` - Add highlight effect

NEW DEPENDENCIES FOR PHASE 2

Install these packages:

```
npx expo install @react-native-community/netinfo  
npx expo install expo-camera  
npx expo install expo-location
```

Already have from Phase 1:

- `expo-image-picker`
- `@react-native-async-storage/async-storage`
- `axios`

FILE STRUCTURE ADDITIONS FOR PHASE 2

New Files to Create:

```
/src/
├── services/ # API layer (you added chatService.js)
│   ├── apiService.js # Base API wrapper with retry logic
│   ├── issueService.js # Issue CRUD operations
│   ├── complaintService.js
│   ├── dashboardService.js
│   └── authService.js
├── components/
│   ├── common/
│   │   ├── OfflineBanner.js # NEW - Offline status banner
│   │   ├── FilterChip.js # NEW - Active filter chips
│   │   └── SearchBar.js # NEW - Enhanced search
│   ├── chat/
│   │   ├── PhotoUploadButton.js # NEW - Camera button
│   │   └── PhotoPreview.js # NEW - Photo preview
│   └── modals/
│       ├── CameraModal.js # NEW - Camera interface
│       └── LocationPermissionModal.js # NEW - Location permission
├── hooks/
│   ├── useNetworkStatus.js # NEW - Network monitoring
│   └── useDebounce.js # NEW - Search debouncing
├── utils/
│   ├── network.js # NEW - Retry logic, network utils
│   ├── photoStorage.js # NEW - Photo storage helpers
│   └── location.js # VERIFY EXISTS - GPS utilities
```

Files to Modify:

- All list screens - Add RefreshControl
- ChatbotScreen - Add photo upload button, integrate location
- IssueDetailScreen - Add highlight effect
- ComplaintDetailScreen - Add highlight effect
- All components - Check network status for offline banner
- Redux slices - Enhance with new actions

REDUX ENHANCEMENTS FOR PHASE 2

offlineSlice.js (Full Implementation):

State:

```
{  
  isOnline: true,  
  pendingActions: [],  
  lastSyncTime: null  
}
```

Actions:

- `setOnlineStatus(isOnline)`
- `addPendingAction(action)`
- `removeSyncedAction(actionId)`
- `incrementRetryCount(actionId)`
- `clearSyncedActions()`

Thunks:

- `syncPendingActions()` - Processes queue when online

issuesSlice.js Additions:

State additions:

```
{  
  filters: {  
    status: [],  
    priority: null,  
    siteIds: [],  
    dateRange: {},  
    deadline: null  
  },  
  searchQuery: ""  
}
```

Actions:

- `applyFilters(filters)`
- `clearFilters()`
- `setSearchQuery(query)`
- `refreshIssues()` - For pull-to-refresh

Selectors:

- `selectFilteredAndSearchedIssues` - Combined filtering

complaintsSlice.js Additions:

Same filter/search structure as issuesSlice.

TESTING CHECKLIST FOR PHASE 2 MVP

Feature 3: Offline Mode

- Offline banner appears when connection lost
- Banner disappears when reconnected (optional: brief green "Back online" message)
- No action queuing - users retry actions manually
- Cached data displayed with timestamp when offline
- Simple network status indicator only

Feature 6: Pull-to-Refresh

- Pull gesture triggers refresh on all list screens
- Spinner appears and data updates
- Works seamlessly when online
- Disabled when offline (shows toast instead)
- Cooldown prevents spam refreshing

Feature 5: Filters & Search

- Search debounces (500ms)
- Filter modal opens and applies filters
- Filter chips display and are removable
- Clear All removes all filters
- Results count updates correctly

Feature 4: Network Retry

- Failed requests retry automatically
- Toast shows retry attempts
- Exponential backoff implemented
- After max retries, shows error message
- Form data preserved during retry

Feature 7: Photo Upload

- Camera icon shows for Supervisor/Solver only
- Take Photo opens camera
- Choose Gallery opens picker

- Photo preview shows in chat
- Upload progress bar displays
- Multiple photos (max 3) supported
- File validation works (size, type)

Feature 8: Location Capture

- LocationPermissionModal shows first time
- Permission request works
- GPS coordinates captured after photo upload
- Location Captured badge shows
- Site verification calculates distance
- Warning if far from site
- Location stored with photo

Feature 2: Internal Navigation

- Tapping notification banner navigates correctly
- Issue notification → IssueDetailScreen
- Complaint notification → ComplaintDetailScreen
- Highlight effect shows and fades

IMPLEMENTATION PRIORITIES

1. Feature 3 (Offline Mode) + Feature 4 (Network Retry)
 - Core infrastructure for reliability
 - Network utilities
 - Banner components
2. Feature 6 (Pull-to-Refresh) + Feature 5 (Filters)
 - Enhanced UX on all list screens
 - Filter modal and chips
 - Debounced search
3. Feature 7 (Photo Upload) + Feature 8 (Location)
 - Camera integration
 - Location permissions
 - GPS capture and verification
4. Feature 2 (Internal navigation) + Testing
 - In-app navigation

- Highlight effects
 - Full integration testing
-

CRITICAL NOTES

- **No TypeScript** - Pure JavaScript only
 - **Mock Backend** - All API calls to mock services (no real server)
 - **Photo Storage** - Local AsyncStorage for Phase 2 (real server in Phase 3)
 - **Compression Later** - 10MB file size limit, compression in Phase 3
 - **Push Notifications** - NOT in this MVP (Phase 3)
 - **Shimmer Loading** - NOT in this MVP (Phase 3)
 - **PDF Export** - NOT in this MVP (Phase 3)
-

SUMMARY

This Phase 2 MVP focuses on **7 production-critical features** that make the app reliable in industrial environments with poor connectivity:

1. **Offline Mode** - Works without internet, queues actions
2. **Pull-to-Refresh** - Refresh data on dashboard and lists
3. **Advanced Filters** - Comprehensive filtering and search
4. **Network Retry** - Automatic retry with backoff
5. **Photo Upload** - Real camera integration
6. **Location Capture** - GPS verification at sites
7. **Internal Navigation** - In-app notification navigation

All features integrate with existing Phase 1 foundation. No breaking changes. Focus on reliability, offline capability, and production UX.

Build Phase 2 MVP Now.